

Einstein/CTensorNP

Setup

```
In[*]:= << mGRG`Einstein`
```

```
In[*]:= $PreRead = ReplaceAll[#,  
  expr_String => StringReplace[expr, "EP" -> "mGRG`Einstein`Private"]] &;
```

BasisNP: Calculate Newman-Penrose Frame

```
In[*]:= BasisNP::usage
```

```
Out[*]:=
```

```
BasisNP[metric, simpCmd] generates a Newman-Penrose  
null tetrad {l, n, m, m-bar} from a 4x4 metric matrix.  
'simpCmd' is an optional simplification function.
```

Metric을 인자로 하여 Newman-Penrose 기준 벡터를 얻는다. 옵션으로 simplification을 위한 함수가 될 수 있다. (See Chandrasekhar, pp 134)

```
In[*]:= coSys = {t, r, θ, ϕ};  
metric =  
  {{-1 +  $\frac{2M}{r}$ , 0, 0, 0}, {0,  $\frac{1}{1 - \frac{2M}{r}}$ , 0, 0}, {0, 0, r2, 0}, {0, 0, 0, r2 Sin[θ]2}};
```

```
In[*]:= nullVs = BasisNP[metric, Simplify]; MatrixForm[nullVs]
```

```
Out[*]//MatrixForm=
```

$$\begin{pmatrix} \frac{r}{2M-r} & 1 & 0 & 0 \\ -\frac{1}{2} & -\frac{1}{2} + \frac{M}{r} & 0 & 0 \\ 0 & 0 & \frac{\sqrt{\frac{1}{r^2}}}{\sqrt{2}} & \frac{\sqrt{-\frac{\text{Csc}[\theta]^2}{r^4}}}{\sqrt{2} \sqrt{\frac{1}{r^2}}} \\ 0 & 0 & \frac{\sqrt{\frac{1}{r^2}}}{\sqrt{2}} & -\frac{\sqrt{-\frac{\text{Csc}[\theta]^2}{r^4}}}{\sqrt{2} \sqrt{\frac{1}{r^2}}} \end{pmatrix}$$

BasisNP의 결과값을 확인한다.

```
In[*]:= MatrixForm[Table[nullVs[[i]] . metric . nullVs[[j]], {i, 4}, {j, 4}] // Simplify]
```

```
Out[*]//MatrixForm=
```

$$\begin{pmatrix} 0 & -1 & 0 & 0 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

```
In[*]:= nullVs = PowerExpand[nullVs]
```

```
Out[*]=
```

$$\left\{ \left\{ \frac{r}{2M-r}, 1, 0, 0 \right\}, \left\{ -\frac{1}{2}, -\frac{1}{2} + \frac{M}{r}, 0, 0 \right\}, \right. \\ \left. \left\{ 0, 0, \frac{1}{\sqrt{2}r}, \frac{i \operatorname{Csc}[\theta]}{\sqrt{2}r} \right\}, \left\{ 0, 0, \frac{1}{\sqrt{2}r}, -\frac{i \operatorname{Csc}[\theta]}{\sqrt{2}r} \right\} \right\}$$

RotateNP: Basis Transformation

```
In[*]:= RotateNP::usage
```

```
Out[*]=
```

`RotateNP[nullVectors, classN, p1, p2, simpCmd]` performs a Lorentz rotation on the given 'nullVectors'. 'classN' specifies the rotation type (1: null rotation about l, 2: null rotation about n, 3: spin-boost). 'p1' and 'p2' are rotation parameters. 'simpCmd' is an optional simplification function.

Newman-Penrose 기준 벡터를 변환한다. 첫 번째 인자는 기준 벡터이고, 두 번째 인자는 변환 Class이다. 세 번째와 네 번째 인자는 변환 변수이다. 옵션으로 simplification을 위한 함수가 될 수 있다.

```
In[*]:= RotateNP[nullVs, 3, \frac{r}{r-2M}, 0, Simplify]
```

```
Out[*]=
```

$$\left\{ \left\{ -1, 1 - \frac{2M}{r}, 0, 0 \right\}, \left\{ \frac{r}{4M-2r}, -\frac{1}{2}, 0, 0 \right\}, \right. \\ \left. \left\{ 0, 0, \frac{1}{\sqrt{2}r}, \frac{i \operatorname{Csc}[\theta]}{\sqrt{2}r} \right\}, \left\{ 0, 0, \frac{1}{\sqrt{2}r}, -\frac{i \operatorname{Csc}[\theta]}{\sqrt{2}r} \right\} \right\}$$

Initialization of Component Tensors in NP Formalism

```
In[*]:= InitCTensorNP::usage
```

```
Out[*]=
```

`InitCTensorNP[coSys, nullVectors, opts]` initializes the environment for Newman-Penrose formalism calculations. 'coSys' is a list of 4 coordinates, and 'nullVectors' is the 4x4 null tetrad matrix {l, n, m, m-bar}. Options can be used to control re-initialization and simplification.

좌표계와 기준 벡터를 입력으로 받아 Newman-Penrose 방식으로 성분 텐서를 초기화 한다. 이 함수는 SpinCoefficients와 곡률 텐서를 자동적으로 계산하고, CoordinateBasisFlag과 KdeltaFlag을 Off 시킨다. 옵션으로 SimplifyMore가 있다.

```
In[*]:= coSys = {t, r,  $\theta$ ,  $\phi$ };
nullVs = {{ $\frac{r}{2M-r}$ , 1, 0, 0}, {- $\frac{1}{2}$ , - $\frac{1}{2} + \frac{M}{r}$ , 0, 0},
  {0, 0,  $\frac{1}{\sqrt{2}r}$ ,  $\frac{i \text{Csc}[\theta]}{\sqrt{2}r}$ }, {0, 0,  $\frac{1}{\sqrt{2}r}$ , - $\frac{i \text{Csc}[\theta]}{\sqrt{2}r}$ }};
```

```
In[*]:= Timing[InitCTensorNP[coSys, nullVs, SimplifyMore → True]]
```

```
Out[*]=
```

```
{0.052576, Null}
```

SpinCoefficients, PhiNP, PsiNP, PetrovType의 값을 보여 준다.

```
In[*]:= Show[SpinCoefficients]
```

$\epsilon = \kappa = \lambda = \nu = \pi = \sigma = \tau = 0$

$$\alpha = \frac{\text{Cot}[\theta]}{2\sqrt{2}r}$$

$$\beta = -\frac{\text{Cot}[\theta]}{2\sqrt{2}r}$$

$$\gamma = -\frac{M}{2r^2}$$

$$\mu = \frac{-2M+r}{2r^2}$$

$$\rho = \frac{1}{r}$$

```
In[*]:= Show[PhiNP]
```

$\Phi_{00} = \Phi_{01} = \Phi_{02} = \Phi_{11} = \Phi_{12} = \Phi_{22} = \Lambda = 0$

```
In[*]:= Show[PsiNP]
```

$\Psi_0 = \Psi_1 = \Psi_3 = \Psi_4 = 0$

$$\Psi_2 = -\frac{M}{r^3}$$

```
In[*]:= Show[PetrovType]
```

Petrov Type: D

```
In[*]:= {WeylCD[-1, -2, -1, -2],
  RiemannCD[-1, -2, -1, -2], RicciCD[-1, -1], ScalarCD[]}
```

```
Out[*]=
```

$$\left\{-\frac{2M}{r^3}, -\frac{2M}{r^3}, 0, 0\right\}$$

Clear Component Tensors in NP Formalism

```
In[*]:= ClearCTensorNP[]
```

In[*]:= {WeylCD[-1, -2, -1, -2], PsiNP[0], PhiNP[0, 1], LambdaNP}

Out[*]= {C₁₂₁₂, PsiNP[0], PhiNP[0, 1], LambdaNP}

In[*]:= {RicciCD[-1, -1], ScalarCD[], RiemannCD[-1, -2, -1, -2]}

Out[*]= {R₁₁, R, R₁₂₁₂}

In[*]:= {EP`epsilonNP, EP`kappaNP, EP`piNP, EP`gammaNP, EP`tauNP, EP`nuNP}

Out[*]= {mGRG`Einstein`Private`epsilonNP, mGRG`Einstein`Private`kappaNP,
mGRG`Einstein`Private`piNP, mGRG`Einstein`Private`gammaNP,
mGRG`Einstein`Private`tauNP, mGRG`Einstein`Private`nuNP}

In[*]:= {EP`betaNP, EP`sigmaNP, EP`lambdaNP, EP`rhoNP, EP`muNP, EP`alphaNP}

Out[*]= {mGRG`Einstein`Private`betaNP, mGRG`Einstein`Private`sigmaNP,
mGRG`Einstein`Private`lambdaNP, mGRG`Einstein`Private`rhoNP,
mGRG`Einstein`Private`muNP, mGRG`Einstein`Private`alphaNP}

In[*]:= {GammaCD[-1, -1, -1], GammaCD[-1, -1, 1]}

Out[*]= {Γ₁₁₁, Γ₁₁¹}

In[*]:= {Structuref[-1, -2, -1], Structuref[-1, -2, 1]}

Out[*]= {f₁₂₁, f₁₂¹}

In[*]:= {EP`inverseBasisMatrix, mGRG`STensor`Private`basisMatrix[DefaultKind]}

Out[*]= {mGRG`Einstein`Private`inverseBasisMatrix,
mGRG`STensor`Private`basisMatrix[Latin]}

In[*]:= {Metricg[-1, -1], Metricg[1, 1]}

Out[*]= {g₁₁, g¹¹}

In[*]:= {EP`nDimension, GetDimension[], GetCoordinates[DefaultKind]}

Out[*]= {mGRG`Einstein`Private`nDimension,
GetDimension[Latin], GetCoordinates[Latin]}

```
In[*]:= {mGRG`STensor`Private`flagTable[InitCTensorFlag],  
          mGRG`STensor`Private`flagTable[EvaluateBDFlag]}
```

```
Out[*]= {False, False}
```

```
In[*]:= {EP`defaultCTensor[EP`simpMethod]}
```

```
Out[*]= {mGRG`Einstein`Private`defaultCTensor[mGRG`Einstein`Private`simpMethod]}
```